

Spécifications techniques

Projet « MultiPrep » - application desktop Windows de préparation et fusion de documents PDF

Version documentée : v1.1.0

Date : 11 mai 2026

Sommaire

Partie	Contenu
<u>1. Contexte et objectif</u>	Pourquoi l'outil existe et quel problème il résout.
<u>2. Périmètre fonctionnel</u>	Fonctions couvertes par la version actuelle.
<u>3. Choix technologiques</u>	Python, version, bibliothèques et justification.
<u>4. Architecture</u>	Organisation du projet et rôle des dossiers.
<u>5. Fonctionnement technique</u>	Flux d'import, de manipulation et de génération PDF.
<u>6. Fichiers et données</u>	Dossiers de travail, exports, temporaires et configuration.
<u>7. Packaging</u>	Construction de l'exécutable Windows avec PyInstaller.
<u>8. Robustesse et sécurité</u>	Validations, limites et confidentialité locale.
<u>9. Maintenance</u>	Mises à jour et évolutions possibles.

1. Contexte et objectif de l'outil

MultiPrep est une application desktop Windows développée en Python avec PySide6. Son objectif est de faciliter la préparation d'un dossier PDF selon les règles de renommage de la MDPH27 final à partir de plusieurs sources : fichiers PDF, images JPG/PNG, captures d'écran collées depuis le presse-papiers et PDF de séparation prédéfinis.

L'outil répond à un besoin opérationnel : assembler rapidement des documents dans un ordre précis, insérer des séparateurs lisibles, contrôler visuellement les pages importées, puis produire un PDF final nommé selon une convention métier.

Besoin	Réponse technique dans MultiPrep
Assembler plusieurs documents	Import PDF/images, conversion en PageItem, puis fusion finale avec pypdf.
Contrôler visuellement le dossier	Miniatures générées avec PyMuPDF et affichées dans une grille PySide6.
Réorganiser rapidement les pages	QListWidget en mode IconMode avec drag and drop personnalisé.
Ajouter des séparateurs métier	Lecture des PDF placés dans le dossier separateurs/.
Produire un fichier prêt à l'emploi	Export automatique dans exports/ avec nom normalisé.

2. Périmètre fonctionnel

Fonction	Description	Fichiers concernés
Import PDF/images	Import par bouton ou glisser-déposer de PDF, JPG, JPEG et PNG.	main_window_actions.py, pdf_service.py, drop_service.py
Collage de capture d'écran	Collage d'une image depuis le presse-papiers avec Ctrl + V ou clic droit.	clipboard_service.py, main_window_actions.py
Miniatures	Affichage de chaque page sous forme de carte, avec une couleur commune pour toutes les pages provenant du même document importé.	thumbnail_service.py, page_thumbnail.py
Réorganisation	Déplacement d'une ou plusieurs pages avec placeholder visuel.	page_grid_list.py, page_board.py
Rotation	Rotation gauche/droite appliquée dans l'aperçu et le PDF final.	main_window_actions.py, page_preview.py, pdf_service.py
Séparateurs	Insertion de pages PDF de séparation avant/après une page ou à la fin.	dialogs.py, pdf_service.py
Export PDF	Fusion dans l'ordre affiché avec nom de fichier normalisé.	name_service.py, pdf_service.py
Écran résultat	Affichage du fichier généré et zone draggable vers une autre application.	result_view.py, dialogs.py

3. Choix technologiques

Choix du langage et de la version

Choix	Justification
Python	Adapté à l'automatisation locale, à la manipulation de fichiers, aux interfaces desktop et au traitement PDF. Le code reste lisible pour une maintenance simple.
Python 3.11 ou plus récent	Base moderne et stable. Compatible avec les annotations de type utilisées dans le projet et avec les versions récentes de PySide6, PyMuPDF, pypdf et PyInstaller.
Application desktop	Plus cohérente qu'une application web car l'outil manipule des fichiers locaux, le presse-papiers Windows, le drag and drop et Outlook.
Pas de base de données	Les données persistantes sont très limitées. settings.json suffit pour mémoriser la dernière date utilisée.

Bibliothèques externes

Bibliothèque	Version	Rôle	Pourquoi ce choix
PySide6	>= 6.11	Interface graphique desktop : fenêtre, widgets, signaux, menus, presse-papiers, drag and drop.	Qt fournit une base robuste pour une application Windows riche sans serveur ni navigateur.
PyMuPDF	>= 1.27	Ouverture PDF, rendu des pages en images, génération de miniatures, conversion image vers PDF temporaire.	Très adapté au rendu rapide de pages PDF pour l'aperçu visuel.
pypdf	>= 4.0	Lecture des PDF sources, copie des pages, rotation et écriture du PDF final.	Simple et fiable pour l'assemblage final des pages sans s'occuper du rendu graphique.

Bibliothèques standard Python

Module	Utilisation dans MultiPrep
pathlib	Manipulation claire des chemins : assets, exports, temp, separateurs, settings.json.
shutil	Nettoyage des dossiers temporaires, notamment temp/cache et temp/mail_attachments.
json	Lecture et écriture de settings.json.
dataclasses	Utilisé dans multiprep/models/page_model.py pour créer les modèles de données principaux : SourceDocument décrit un fichier source, PageItem décrit une page affichée/manipulée dans l'interface, et SeparatorOption décrit un PDF de séparation disponible dans le dossier separateurs/.
datetime.date	Validation de la date saisie et formatage du nom de sortie.
re / unicodedata	Nettoyage des noms et prénoms pour générer des noms de fichiers stables.
ctypes	Déclaration d'un identifiant d'application Windows pour améliorer l'intégration système.
copy	Copie des pages PDF avant rotation et ajout dans le fichier final.

4. Architecture du projet

Le projet est organisé autour d'un package principal `multiprep/`. Cette architecture sépare l'interface, la logique métier, les modèles et les utilitaires. Elle évite de regrouper toute la logique dans un seul fichier et rend le projet plus facile à expliquer, maintenir et faire évoluer.

Dossier / fichier	Rôle	Pourquoi il existe
run.py	Point d'entrée minimal.	Permet de lancer l'application en développement et sert d'entrée à PyInstaller.
multiprep/ui/	Interface PySide6.	Contient les vues, la fenêtre principale, la grille, les dialogues et les styles.
multiprep/services/	Logique métier.	Regroupe PDF, presse-papiers, drag and drop, paramètres et noms de fichiers.
multiprep/models/	Modèles de données.	Structure les objets partagés entre interface et services.
multiprep/utils/	Constantes et utilitaires.	Centralise chemins et couleurs pour éviter la duplication.
assets/	Ressources graphiques.	Stocke l'icône utilisée par l'application et l'exécutable.
separateurs/	PDF de séparation.	Permet à l'utilisateur d'ajouter des séparateurs sans modifier le code.
exports/	Sorties utilisateur.	Contient les PDF finaux générés.
temp/	Travail temporaire.	Contient miniatures, captures, PDF temporaires et pièces jointes Outlook.
dist/	Sortie PyInstaller.	Contient MultiPrep.exe.
build/	Intermédiaires PyInstaller.	Dossier généré, non utile pour l'utilisateur final ni pour comprendre le code.

Fichiers Python principaux

Fichier	Responsabilité
app_entry.py	Crée QApplication, applique l'icône et affiche MultiPrepWindow.
main_window.py	Assemble la fenêtre, les services, les champs et les vues.
main_window_actions.py	Contient les actions utilisateur : importer, coller, supprimer, tourner, générer.
pdf_service.py	Gère import PDF, images, séparateurs, miniatures et fusion finale.
drop_service.py	Interprète les fichiers déposés, y compris les pièces jointes Outlook.
clipboard_service.py	Transforme une image du presse-papiers en page PDF temporaire.
page_grid_list.py	Gère la grille de pages, la sélection multiple et le drag and drop.
page_model.py	Définit SourceDocument, PageItem et SeparatorOption.
name_service.py	Normalise nom/prénom et construit le nom du PDF final.
settings_service.py	Lit et écrit les paramètres utilisateur dans settings.json.

5. Fonctionnement technique

Étape	Déroulement technique
Démarrage	run.py appelle main() dans app_entry.py. QApplication est créée, puis MultiPrepWindow est affichée.
Import PDF	Un SourceDocument est créé. PdfService ouvre le PDF avec PyMuPDF, génère les miniatures, puis retourne des PageItem.
Import image	L'image est convertie en PDF temporaire dans temp/cache, puis représentée par un PageItem.
Collage capture	ClipboardService lit QApplication.clipboard(), sauvegarde l'image en PNG, puis délègue à PdfService.
Réorganisation	PageGridListWidget conserve l'ordre visuel des PageItem après drag and drop.
Insertion séparateur	PdfService lit les PDF du dossier separateurs/ et retourne les pages à insérer dans la liste.
Génération PDF	generate_pdf valide les champs, name_service construit le nom, puis PdfService.merge écrit le PDF final.
Résultat	ResultView affiche le fichier et DraggableFile permet de le glisser vers une autre application.

6. Gestion des fichiers et données

Élément	Type	Gestion
settings.json	Configuration utilisateur	Mémoire la dernière date utilisée. Lu au démarrage, écrit après génération réussie.
temp/cache/	Temporaire	Miniatures, captures converties et PDF temporaires. Nettoyé par PdfService.

temp/mail_attachments/	Temporaire	Pièces jointes Outlook extraites. Nettoyé par drop_service.py.
exports/	Résultat utilisateur	Reçoit les PDF finaux générés.
separateurs/	Entrée utilisateur	Contient les PDF de séparation proposés dans l'interface.
build/	Généré	Intermédiaire PyInstaller. Ignoré par Git.
dist/	Généré	Contient l'exécutable final MultiPrep.exe.

7. Packaging Windows avec PyInstaller

Élément du .spec	Valeur	Rôle
Analysis	['run.py']	Indique le point d'entrée à analyser.
datas	assets/multiprep-logo.ico	Ajoute le fichier icône dans les fichiers embarqués avec l'application. Cela permet au code Python de retrouver l'icône au lancement, même une fois l'application transformée en .exe.
name	MultiPrep	Nom de l'exécutable généré.
console	False	Produit une application fenêtre sans console noire.
icon	assets\\multiprep-logo.ico	Définit l'icône Windows affichée sur l'exécutable.
dist/	MultiPrep.exe	Sortie distribuable à l'utilisateur.

PyInstaller est adapté ici car l'objectif est de distribuer une application Windows simple à lancer, sans demander à l'utilisateur d'installer Python, les dépendances ou un environnement virtuel.

8. Robustesse, sécurité et limites

Sujet	Mesure / limite
Validation utilisateur	La génération est bloquée si aucune page n'est importée, si la date est invalide ou si nom/prénom sont absents.
Fichiers invalides	Les erreurs d'import sont signalées par QMessageBox et n'arrêtent pas toute l'application.
Nettoyage	Les dossiers temporaires sont nettoyés au démarrage et à la fermeture normale.
Confidentialité	Les documents restent locaux. Aucun fichier n'est envoyé à un serveur externe.
Dépendances	Le projet limite les bibliothèques externes à trois dépendances principales.
Limite PDF	Les PDF corrompus, protégés ou très atypiques peuvent échouer à l'import ou à la fusion.
Cible système	L'application vise principalement Windows, notamment pour Outlook et le drag and drop système.

9. Maintenance et évolutions possibles

Type de maintenance	Description
Corrective	Correction de bugs d'import, de rendu, de glisser-déposer ou de génération PDF.
Technique	Mise à jour de PySide6, PyMuPDF, pypdf, PyInstaller et vérification de compatibilité Windows.

Documentaire	Mise à jour du README, de la documentation HTML et de ce document lorsque l'architecture évolue.
Évolutive	Profils de séparateurs, aperçu multi-pages, autres conventions de nommage, journalisation, tests automatisés.

Ordre conseillé pour comprendre le code

Ordre	Fichier	Pourquoi le lire
1	run.py	Voir le point d'entrée.
2	multiprep/ui/app_entry.py	Comprendre la création de QApplication et de la fenêtre.
3	multiprep/ui/main_window.py	Comprendre l'assemblage principal.
4	multiprep/ui/main_window_actions.py	Suivre les scénarios utilisateur.
5	multiprep/services/pdf_service.py	Comprendre le cœur du traitement PDF.
6	multiprep/models/page_model.py	Comprendre les données manipulées.
7	multiprep/ui/page_board.py et page_grid_list.py	Comprendre la grille, la sélection et le drag and drop.
8	multiprep/services/drop_service.py et clipboard_service.py	Comprendre les interactions Windows et le presse-papiers.